

Homework 1 Report

Huffman

Encoding Approach

The approach I took for the Huffman encoding was as follows: Before writing any code, I first wrote down the steps as they were presented in lecture on a piece of paper and worked through a small example using the input "hello world." Doing the problem by hand helped me get a feel for each step involved in the algorithm and gave me a better understanding of how everything fit together before I started implementing it

Handwritten Example

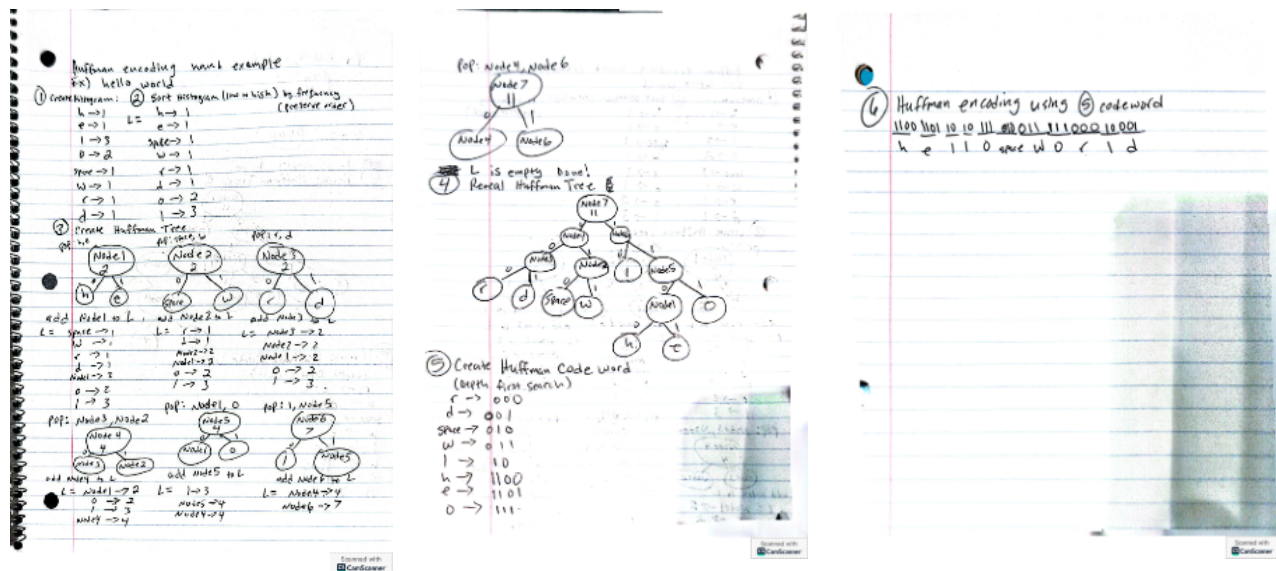


Figure 1. The above illustrates the step-by-step handwritten Huffman encoding process used to develop an intuitive understanding of the algorithm to guide the design and implementation of the code.

Takeaway

The three main things I learned from the handwritten example were that I first needed to build a histogram of the input symbols. Then, from the histogram, I needed a data structure that, when removing from and adding to the sorted histogram variable L, would preserve its sorted property. Lastly, I needed to create my own Node struct to encapsulate the current symbol, frequency count, and the left and right child node indices.

The next major hurdle was to traverse the tree. For this I used the depth first search traversal to build the code word for each symbol with a runtime of $O(n)$ where n is the number of nodes in the tree since it has to visit each node once.

Decoding Example

⑦ Decoding

1100110110111010011110001001

I need to know: The original histogram and code words.

	code words
$h \rightarrow 1$	$r \rightarrow 000$
$e \rightarrow 1$	$d \rightarrow 001$
$l \rightarrow 0$	$space \rightarrow 010$
$o \rightarrow 2$	$w \rightarrow 011$
$space \rightarrow 1$	$l \rightarrow 10$
$w \rightarrow 1$	$h \rightarrow 1100$
$r \rightarrow 1$	$e \rightarrow 1101$
$d \rightarrow 1$	$o \rightarrow 111$

Original histogram (unknown)

Now I can build Huffman tree from the histogram.

Which is the same as step (4) on previous page.

Using the above mentioned tree I can read the encoded stream of bits one by one until I reach a terminal (or leaf node)

So $1100 \rightarrow h$. Start back at the root and continue the process.

Code Output

The screenshots below show the results obtained using the helloworld.txt input file. The implementation successfully encoded the input data and correctly decoded it back to its original form, verifying the correctness of the Huffman coding algorithm. However, the generated code.txt file was larger than the original uncompressed input file. I had to

research why and found this result is expected for small files because the overhead required to store the Huffman tree (the histogram) can outweigh the savings achieved through the original encoding. Thus, the Huffman coding does not guarantee compression for every input [1].

```
ubuntu@e2a35d3d0622: /workspaces/wes237b/assignment-1/homework/huffman$ make
g++ -Og -g -Iinclude src/huffman.cpp src/main.cpp -o huffman
ubuntu@e2a35d3d0622: /workspaces/wes237b/assignment-1/homework/huffman$ ./huffman helloworld.txt code.txt output.txt
WES237B Assignment 1
w: 1
r: 1
h: 1
e: 1
d: 1
: 1
o: 2
l: 3

Huffman Tree Codes:
d = 000
h = 001
e = 010
l = 011
w = 100
r = 110
o = 111
SUCCESS
ubuntu@e2a35d3d0622: /workspaces/wes237b/assignment-1/homework/huffman$ ls -la
total 628
drwxrwxr-x 6 ubuntu ubuntu 4096 Aug 1 20:41 .
drwxrwxr-x 4 ubuntu ubuntu 4096 Aug 1 04:10 ..
-rw-rw-r-- 1 ubuntu ubuntu 25 Jul 12 19:20 .gitignore
drwxrwxr-x 2 ubuntu ubuntu 4096 Aug 1 20:41 .vscode
-rw-rw-r-- 1 ubuntu ubuntu 376 Jul 12 19:20 Makefile
-rw-rw-r-- 1 ubuntu ubuntu 52 Aug 1 20:39 code.txt
-rw-rw-r-- 1 ubuntu ubuntu 11 Jul 22 04:58 helloworld.txt
-rwxr-xr-x 1 ubuntu ubuntu 435760 Aug 1 20:38 huffman
drwxrwxr-x 2 ubuntu ubuntu 4096 Jul 12 19:20 include
-rw-rw-r-- 1 ubuntu ubuntu 34187 Jul 12 19:20 input.txt
drwxrwxr-x 2 ubuntu ubuntu 4096 Jul 12 19:20 obj
-rw-rw-r-- 1 ubuntu ubuntu 11 Aug 1 20:39 output.txt
drwxrwxr-x 2 ubuntu ubuntu 4096 Jul 12 19:20 src
-rw-rw-r-- 1 ubuntu ubuntu 157781 Aug 1 20:41 testoutput.png
ubuntu@e2a35d3d0622: /workspaces/wes237b/assignment-1/homework/huffman$
```

Homework Output

```

c-main.cpp c-huffman.cpp M 5 results.txt x 1 README.md
homedow > huffman > 5 results.txt
1
2
3 : 1
4 L: 4
5 : 12
6 y: 19
7 U: 21
8 Q: 22
9 J: 38
10 E: 33
11 F: 45
12 I: 46
13 M: 49
14 A: 49
15 O: 62
16 V: 72
17 S: 73
18 C: 76
19 N: 95
20 P: 151
21 h: 179
22 f: 186
23 q: 334
24 b: 375
25 v: 384
26 g: 398
27 p: 666
28 ,: 666
29 d: 714
30 : 773
31 c: 1184
32 a: 1154
33 m: 1234
34 r: 1515
35 n: 1649
36 l: 1722
37 at: 2144
38 t: 2284
39 s: 2352
40 u: 2622
41 i: 2696
42 ex: 2366
43 : 4999
44
45 Huffman Tree Codes:
46 r = 0000
47 - = 00010
48 v = 000110
49 g = 000111
50 e = 001
51 n = 0100
52 l = 0101
53 a = 0110
54 t = 0111
55 c = 10000
56 o = 10001
57 s = 1001
58 , = 101
59 M = 110000000
60 A = 110000001
61 D = 110000010
62 j = 1100000110
63 E = 1100000111
64 V = 110000100
65 S = 110000101
66 P = 11000011
67 C = 110001000
68
69 = 1100010010000
70 L = 1100010010001
71 j = 110001001001
72 y = 11000100101
73 U = 11000100110
74 Q = 11000100111
75 h = 11000101
76 q = 1100011
77 m = 1001
78 u = 1001
79 i = 1110
80 p = 111100
81 , = 111101
82 d = 111110
83 F = 111110000
84 I = 1111110001
85 N = 111111001
86 f = 11111101
87 b = 1111111
88 SUCCESS
89

```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 3
● ubuntu@e2a35d3d0622: /workspaces/wes237b/assignment-1/homework/huffman$ ./huffman input.txt code.txt output.txt >> results.txt
● ubuntu@e2a35d3d0622: /workspaces/wes237b/assignment-1/homework/huffman$ ls -la
total 844
drwxrwxr-x 6 ubuntu ubuntu 4096 Aug 1 21:56 .
drwxrwxr-x 4 ubuntu ubuntu 4096 Aug 1 04:10 ..
-rw-rw-r-- 1 ubuntu ubuntu 25 Jul 12 19:20 .gitignore
drwxrwxr-x 2 ubuntu ubuntu 4096 Aug 1 20:41 .vscode
-rw-rw-r-- 1 ubuntu ubuntu 376 Jul 12 19:20 Makefile
-rw-r--r-- 1 ubuntu ubuntu 18518 Aug 1 21:56 code.txt
-rw-r--r-- 1 ubuntu ubuntu 11 Jul 22 04:58 helloworld.txt
-rwxr-xr-x 1 ubuntu ubuntu 435760 Aug 1 20:59 huffman
drwxrwxr-x 2 ubuntu ubuntu 4096 Jul 12 19:20 include
-rw-rw-r-- 1 ubuntu ubuntu 34187 Jul 12 19:20 input.txt
drwxrwxr-x 2 ubuntu ubuntu 4096 Jul 12 19:20 objs
-rw-r--r-- 1 ubuntu ubuntu 34187 Aug 1 21:56 output.txt
-rw-r--r-- 1 ubuntu ubuntu 836 Aug 1 21:56 results.txt
drwxrwxr-x 2 ubuntu ubuntu 4096 Jul 12 19:20 src
-rw-r--r-- 1 ubuntu ubuntu 157781 Aug 1 20:41 testoutput.png
-rw-r--r-- 1 ubuntu ubuntu 164106 Aug 1 20:46 testoutput2.png
○ ubuntu@e2a35d3d0622: /workspaces/wes237b/assignment-1/homework/huffman$

```


Vector Add

This homework was much easier than the Huffman project. It was mainly about learning how to use the OpenCL API and understanding the overall OpenCL framework. There wasn't nearly as much implementation required compared to the Huffman assignment. Most of my time was spent reading OpenCL in Action: How to Accelerate Graphics and Computation and applying what I learned to the project [2].

What I learned was how to copy host memory to device, invoke OpenCL kernel, Copy results from device to host, free device memory, and write the OpenCL kernel.

Code Output

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 3
bash - vector_add + + + + +

● ubuntu@2a35d3d0622:/workspaces/wes237b/assignment-1/homework/vector_add$ make run
cd ../../helper_lib; make
make[1]: Entering directory '/workspaces/wes237b/assignment-1/helper_lib'
gcc -g -Wall -c -o device.o device.c -I/usr/local/cuda/include -L/usr/local/cuda/lib64 -lOpenCL -lm
gcc -g -Wall -c -o kernel.o kernel.c -I/usr/local/cuda/include -L/usr/local/cuda/lib64 -lOpenCL -lm
gcc -g -Wall -c -o matrix.o matrix.c -I/usr/local/cuda/include -L/usr/local/cuda/lib64 -lOpenCL -lm
gcc -g -Wall -c -o img.o img.c -I/usr/local/cuda/include -L/usr/local/cuda/lib64 -lOpenCL -lm
ar rcs helper_lib.a device.o kernel.o matrix.o img.o
make[1]: Leaving directory '/workspaces/wes237b/assignment-1/helper_lib'
gcc -g -Wall -o solution ../../helper_lib/helper_lib.a main.c -I../../helper_lib -L/usr/local/cuda/include ../../helper_lib/helper_lib.a -lm -L/usr/local/cuda/lib64 -lOpenCL
./solution Dataset/0/input0.raw Dataset/0/input1.raw Dataset/0/input2.raw Dataset/0/input3.raw Dataset/0/output.raw program_1_output.raw program_2_output.raw
=====Starting Program 1=====
Running on:
Platform: QUALCOMM Snapdragon(TM)
Device: QUALCOMM Adreno(TM) 643

!!SOLUTION IS CORRECT!!
Execution time: 243.46ms
=====Finished Program 1=====
=====Starting Program 2=====
Running on:
Platform: QUALCOMM Snapdragon(TM)
Device: QUALCOMM Adreno(TM) 643

!!SOLUTION IS CORRECT!!
Execution time: 21.96ms
=====Finished Program 2=====
./solution Dataset/1/input0.raw Dataset/1/input1.raw Dataset/1/input2.raw Dataset/1/input3.raw Dataset/1/output.raw program_1_output.raw program_2_output.raw
=====Starting Program 1=====
Running on:
Platform: QUALCOMM Snapdragon(TM)
Device: QUALCOMM Adreno(TM) 643

!!SOLUTION IS CORRECT!!
Execution time: 89.32ms
=====Finished Program 1=====
=====Starting Program 2=====
Running on:
Platform: QUALCOMM Snapdragon(TM)
Device: QUALCOMM Adreno(TM) 643

!!SOLUTION IS CORRECT!!

```

```

ubuntu@e2a35d3d0622:/workspaces/wes237b/assignment-1/homework/vector_add$ make run
Execution time: 17.61ms
=====Finished Program 2=====
./solution Dataset/2/input0.raw Dataset/2/input1.raw Dataset/2/input2.raw Dataset/2/input3.raw Dataset/2/output.raw program_1_output.raw program_2_output.raw
=====Starting Program 1=====
Running on:
Platform: QUALCOMM Snapdragon(TM)
Device: QUALCOMM Adreno(TM) 643

!!SOLUTION IS CORRECT!!
Execution time: 89.66ms
=====Finished Program 1=====
=====Starting Program 2=====
Running on:
Platform: QUALCOMM Snapdragon(TM)
Device: QUALCOMM Adreno(TM) 643

!!SOLUTION IS CORRECT!!
Execution time: 17.23ms
=====Finished Program 2=====
./solution Dataset/3/input0.raw Dataset/3/input1.raw Dataset/3/input2.raw Dataset/3/input3.raw Dataset/3/output.raw program_1_output.raw program_2_output.raw
=====Starting Program 1=====
Running on:
Platform: QUALCOMM Snapdragon(TM)
Device: QUALCOMM Adreno(TM) 643

!!SOLUTION IS CORRECT!!
Execution time: 89.37ms
=====Finished Program 1=====
=====Starting Program 2=====
Running on:
Platform: QUALCOMM Snapdragon(TM)
Device: QUALCOMM Adreno(TM) 643

!!SOLUTION IS CORRECT!!
Execution time: 17.43ms
=====Finished Program 2=====
./solution Dataset/4/input0.raw Dataset/4/input1.raw Dataset/4/input2.raw Dataset/4/input3.raw Dataset/4/output.raw program_1_output.raw program_2_output.raw
=====Starting Program 1=====
Running on:
Platform: QUALCOMM Snapdragon(TM)
Device: QUALCOMM Adreno(TM) 643

!!SOLUTION IS CORRECT!!
Execution time: 95.08ms
=====Finished Program 1=====
=====Starting Program 2=====

```

```

ubuntu@e2a35d3d0622:/workspaces/wes237b/assignment-1/homework/vector_add$ make run
Running on:
Platform: QUALCOMM Snapdragon(TM)
Device: QUALCOMM Adreno(TM) 643

!!SOLUTION IS CORRECT!!
Execution time: 20.26ms
=====Finished Program 2=====
./solution Dataset/5/input0.raw Dataset/5/input1.raw Dataset/5/input2.raw Dataset/5/input3.raw Dataset/5/output.raw program_1_output.raw program_2_output.raw
=====Starting Program 1=====
Running on:
Platform: QUALCOMM Snapdragon(TM)
Device: QUALCOMM Adreno(TM) 643

!!SOLUTION IS CORRECT!!
Execution time: 98.44ms
=====Finished Program 1=====
=====Starting Program 2=====
Running on:
Platform: QUALCOMM Snapdragon(TM)
Device: QUALCOMM Adreno(TM) 643

!!SOLUTION IS CORRECT!!
Execution time: 20.96ms
=====Finished Program 2=====
./solution Dataset/6/input0.raw Dataset/6/input1.raw Dataset/6/input2.raw Dataset/6/input3.raw Dataset/6/output.raw program_1_output.raw program_2_output.raw
=====Starting Program 1=====
Running on:
Platform: QUALCOMM Snapdragon(TM)
Device: QUALCOMM Adreno(TM) 643

!!SOLUTION IS CORRECT!!
Execution time: 125.46ms
=====Finished Program 1=====
=====Starting Program 2=====
Running on:
Platform: QUALCOMM Snapdragon(TM)
Device: QUALCOMM Adreno(TM) 643

!!SOLUTION IS CORRECT!!
Execution time: 52.77ms
=====Finished Program 2=====
./solution Dataset/7/input0.raw Dataset/7/input1.raw Dataset/7/input2.raw Dataset/7/input3.raw Dataset/7/output.raw program_1_output.raw program_2_output.raw
=====Starting Program 1=====
Running on:
Platform: QUALCOMM Snapdragon(TM)
Device: QUALCOMM Adreno(TM) 643

```

```
ubuntu@e2a35d3d0622:/workspaces/wes237b/assignment-1/homework/vector_add$ make run
!!SOLUTION IS CORRECT!!
Execution time: 199.53ms
=====Finished Program 1=====
=====Starting Program 2=====
Running on:
Platform: QUALCOMM Snapdragon(TM)
Device: QUALCOMM Adreno(TM) 643

!!SOLUTION IS CORRECT!!
Execution time: 121.20ms
=====Finished Program 2=====
./solution Dataset/8/input0.raw Dataset/8/input1.raw Dataset/8/input2.raw Dataset/8/input3.raw Dataset/8/output.raw program_1_output.raw program_2_output.raw
=====Starting Program 1=====
Running on:
Platform: QUALCOMM Snapdragon(TM)
Device: QUALCOMM Adreno(TM) 643

!!SOLUTION IS CORRECT!!
Execution time: 367.54ms
=====Finished Program 1=====
=====Starting Program 2=====
Running on:
Platform: QUALCOMM Snapdragon(TM)
Device: QUALCOMM Adreno(TM) 643

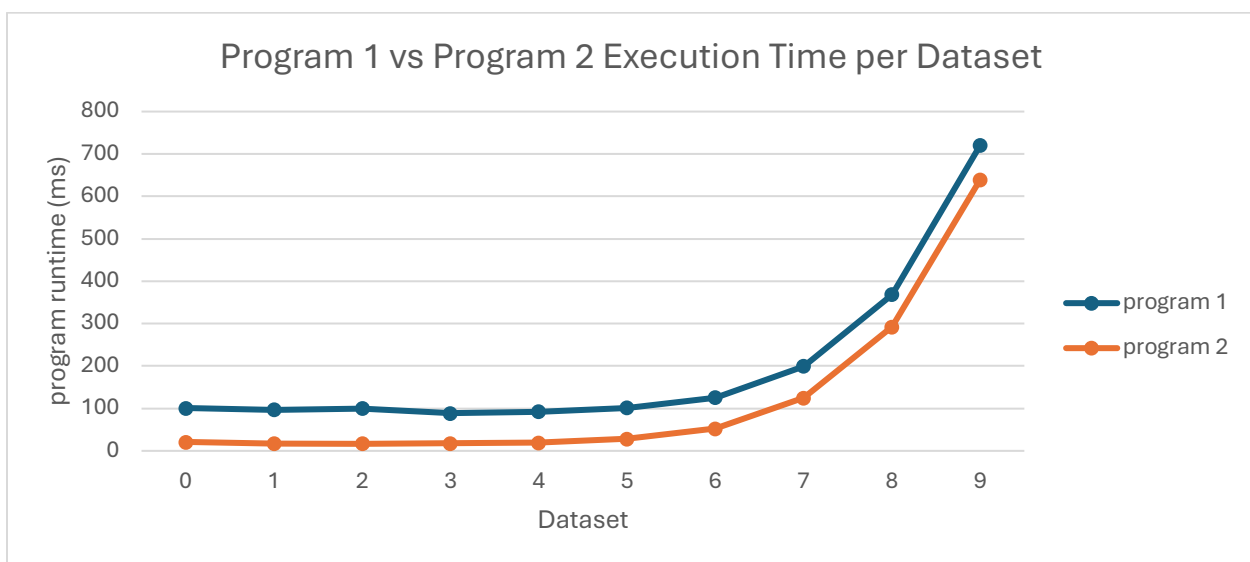
!!SOLUTION IS CORRECT!!
Execution time: 302.14ms
=====Finished Program 2=====
./solution Dataset/9/input0.raw Dataset/9/input1.raw Dataset/9/input2.raw Dataset/9/input3.raw Dataset/9/output.raw program_1_output.raw program_2_output.raw
=====Starting Program 1=====
Running on:
Platform: QUALCOMM Snapdragon(TM)
Device: QUALCOMM Adreno(TM) 643

!!SOLUTION IS CORRECT!!
Execution time: 716.43ms
=====Finished Program 1=====
=====Starting Program 2=====
Running on:
Platform: QUALCOMM Snapdragon(TM)
Device: QUALCOMM Adreno(TM) 643

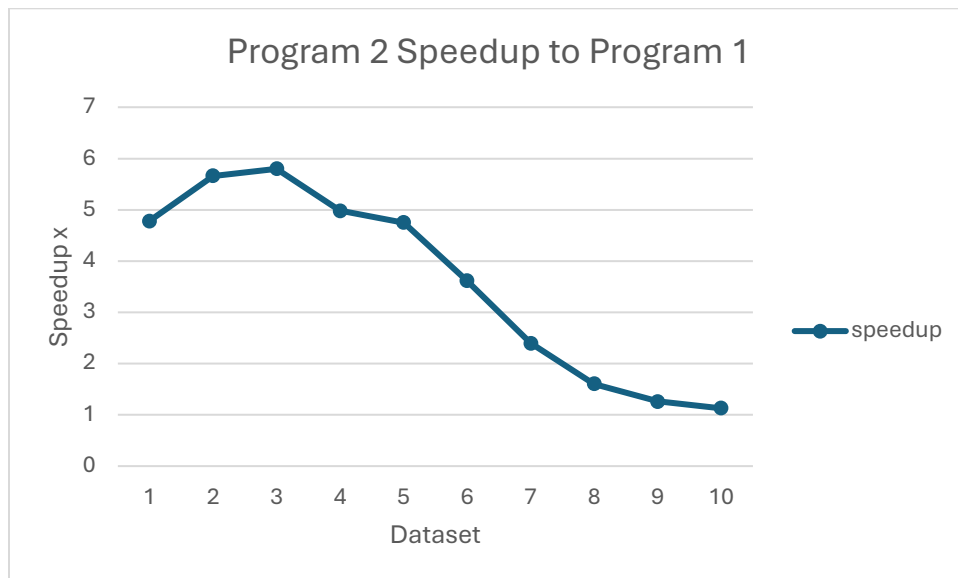
!!SOLUTION IS CORRECT!!
Execution time: 623.00ms
=====Finished Program 2=====
ubuntu@e2a35d3d0622:/workspaces/wes237b/assignment-1/homework/vector_add$
```

Takeaway

Program 2 outperformed Program 1 across all datasets. Program 1 uses three separate kernel calls to add the four vectors, causing repeated GPU memory allocation with the host to device transfers, kernel launches, and device to host transfers, and resource cleanup. Program 2 combines all four vector additions into one single kernel call, reducing this OpenCL overhead. Thus, the results demonstrate that minimizing the kernel launches and memory transfers can impact GPU execution time.



The below figure shows the speedup of Program 2 relative to Program 1. Program 2 achieved a speedup greater than 1X for every dataset, indicating that it consistently executed faster than Program 1. The highest speedup, peaks around dataset 3 with a relative speedup of just under 6X. As the dataset size increased, the speedup steadily decreased for the larger datasets. This indicates that while Program 2 provides a performance improvement for smaller workloads, its relative advantage diminishes as the computational workload grows and the execution times of both implementations become more similar.



Issues with the Autograder on Gradescope

I tested pretty rigorously on my local container and my code works on the datasets provided. Furthermore, I even did the handwritten example of the Huffman encoding/decoding and debugged my code to show the same results as expected. Similarly, the vector add passes the local dataset tests showing no signs of issues. So I think there's some environment discrepancies going on with the autograder server. I will attach my code in the same folder structure as given and can be found using this GitHub link to my class repository.

GitHub: <https://github.com/Math140Instructor/wes237b>

Works Cited

- [1] Massachusetts Institute of Technology, "Lecture 19: Compression and Huffman Coding," Massachusetts Institute of Technology, 2012. [Online]. Available: https://ocw.mit.edu/courses/6-046j-design-and-analysis-of-algorithms-spring-2012/388115265a456321c4a5d19dc9e05281_MIT6_046JS12_lec19.pdf.
- [2] M. Scarpino, OpenCL in Action: How to Accelerate Graphics and Computation, Shelter Island, NY: Manning Publications, 2012.

Lab 1 Outputs

README.md

2026-08-02

Lab 1 Submission

Docker Environment Screenshots

1. OpenCL and device query output.

```
ubuntu@cd0589796b82:/workspaces/wes237b/assignment-1/lab/device_query$ clinfo -l
clinfo: /lib/aarch64-linux-gnu/libOpenCL.so.1: no version information available (required by clinfo)
Platform #0: QUALCOMM Snapdragon(TM)
  -- Device #0: QUALCOMM Adreno(TM) 643
Platform #1: Portable Computing Language
  -- Device #0: cpu-neoverse-n2-cortex-a55
ubuntu@cd0589796b82:/workspaces/wes237b/assignment-1/lab/device_query$ make run
Finding OpenCL Platforms...
Found 2 platforms!
Platform 0
- Name: QUALCOMM Snapdragon(TM)
- Version: OpenCL 3.0 QUALCOMM build: 0838.2
- Vendor: QUALCOMM
- Num Devices: 1
  Device: 0
  - Name: QUALCOMM Adreno(TM) 643
  - Type: GPU
  - Compute units: 2
  - Global memory size: 277556096 bytes (264 MB)
  - Max constant buffer size: 65536 bytes (0 MB)
  - Max local memory size per Compute Unit: 32768 bytes (32 KB)
  - Max Work Item Dimensions: 3
  - Max Work Item size: 1024 x 1024 x 1024
  - Max group size: 1024
Platform 1
- Name: Portable Computing Language
- Version: OpenCL 3.0 PoCL 7.1 Linux, Release, REL0C, LLVM 20.1.8, SLEEF, POCL_DEBUG
- Vendor: The pocl project
- Num Devices: 1
  Device: 0
  - Name: cpu-neoverse-n2-cortex-a55
  - Type: CPU
  - Compute units: 8
  - Global memory size: 4163334144 bytes (3970 MB)
  - Max constant buffer size: 524288 bytes (0 MB)
  - Max local memory size per Compute Unit: 524288 bytes (512 KB)
  - Max Work Item Dimensions: 3
  - Max Work Item size: 4096 x 4096 x 4096
  - Max group size: 4096
```

2. Raytracer sequential output.

```
ubuntu@cd0589796b82:/workspaces/wes237b/assignment-1/lab/raytracer_sequential$ make run
gcc -O2 -std=c99 -o raytracer_sequential main.c lib/geometry/Sphere.c lib/vec_ops.c lib/geometry/Light.c -lm
./raytracer_sequential
Starting Sequential Ray Tracing...
=====
Single-Threaded Implementation on CPU
Total execution time (host + kernel): 2259.226 ms
=====
```

3. Raytracer parallel output

```
ubuntu@cd0589796b82:/workspaces/wes237b/assignment-1/lab/raytracer_parallel$ make
make: Nothing to be done for 'all'.
ubuntu@cd0589796b82:/workspaces/wes237b/assignment-1/lab/raytracer_parallel$ ls
Makefile core kernel.cl lib main.c raytracer_parallel
ubuntu@cd0589796b82:/workspaces/wes237b/assignment-1/lab/raytracer_parallel$ ./raytracer_parallel 0
=====
Device: QUALCOMM Adreno(TM) 643
Total execution time (host + kernel): 304.507 ms
Image titled output_gpu.png has been created/modified and can now be viewed!
=====
```